

Pi Deployment — self-contained EBB instance at ebb.hopto.org

A standalone EBB instance on a Raspberry Pi 4: its own ingestion, its own API, its own copy of the report — independent of the dev machine, reachable at <https://ebb.hopto.org>. This is a test case for full self-hosted deployment, separate from the local Docker setup on the dev machine (`docker-compose.yml` at the repo root — that one stays as-is).

Do not start this until onsite — needs physical access to add the SSD and diagnose the router. See Prerequisites.

Config files for this deployment live in `deploy/pi/` (`docker-compose.yml`, `Caddyfile`) and reuse the repo’s existing `Dockerfile` unchanged.

Prerequisites

1. **Port 443 issue resolved on the D-Link DSL-G225.** `telnet ebb.hopto.org 80` already connects, confirming the forward chain (D-Link → eero → Pi) works for port 80. Port 443 is currently intercepted before it gets that far — most likely the DSL-G225’s own HTTPS admin interface binding 443 independently of its “WAN services” toggle (see: separate troubleshooting thread). Caddy can’t obtain a Let’s Encrypt cert until 443 actually reaches the Pi.
 2. SSD physically attached to the Pi.
 3. SSH access to the Pi from wherever you’re running these commands.
-

1. SSD setup

Identify the SSD (usually `/dev/sda` if it’s the only USB-attached drive — confirm with `lsblk` before partitioning, don’t guess):

```
lsblk
sudo parted /dev/sda --script mklabel gpt mkpart primary ext4 0% 100%
sudo mkfs.ext4 /dev/sda1
sudo mkdir -p /mnt/ebb-ssd
```

Get the UUID and add a persistent fstab entry (more reliable across reboots than `/dev/sda1`, which can shift):

```
sudo blkid /dev/sda1    # copy the UUID=... value
echo 'UUID=<paste-uuid-here> /mnt/ebb-ssd ext4 defaults,noatime 0 2' | sudo tee -a /etc/fstab
sudo mount -a
sudo chown -R "$USER:$USER" /mnt/ebb-ssd
```

`noatime` reduces write wear a little; not critical but cheap to include.

2. Get the project onto the SSD

Either clone fresh (clean history, recommended default):

```
cd /mnt/ebb-ssd
git clone git@bitbucket.org:ebolajoe/cb.ebb.claude.data.git
cd cb.ebb.claude.data
```

...or, if you want to carry over any uncommitted local state from the dev machine too, `rsync` the whole checkout instead (run from the **dev machine**, pushing to the Pi):

```
rsync -avz --exclude venv --exclude __pycache__ --exclude data \
/home/chris/Documents/dev/ebb/Claude/cb.ebb.claude.data/ \
<pi-user>@<pi-host>:/mnt/ebb-ssd/cb.ebb.claude.data/
```

(data excluded here deliberately — transferred separately in step 3, so it isn't double-sent or raced against.)

3. Archive transfer

One-time copy of the current 5.5GB archive so the Pi starts with full history instead of empty. Run from the **dev machine**:

```
rsync -avz --progress \
/home/chris/Documents/dev/ebb/Claude/cb.ebb.claude.data/data/ \
<pi-user>@<pi-host>:/mnt/ebb-ssd/cb.ebb.claude.data/data/
```

Will take a while depending on LAN speed — do this once, before the first ingestion run on the Pi, so nothing is writing to `data/` on either side while it transfers.

4. Python environment

```
cd /mnt/ebb-ssd/cb.ebb.claude.data
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Known risk spot, verify when you get here: `netCDF4` (used by `runner.py` for ERA5 ingestion) needs a prebuilt aarch64 wheel. If `pip install` fails trying to build it from source, install the system dev headers first and retry:

```
sudo apt-get install -y libnetcdf-dev
pip install -r requirements.txt
```

5. API key

Generate a **fresh** key for this deployment — don't reuse the dev machine's `.env`, this is a separate instance with its own secret:

```
cd /mnt/ebb-ssd/cb.ebb.claude.data
python3 -c "import secrets; print('EBB_API_KEY=' + secrets.token_hex(32))" > .env
chmod 600 .env
```

6. Ingestion + report cron

Reuse the existing interactive generator rather than hand-writing cron lines:

```
cd /mnt/ebb-ssd/cb.ebb.claude.data
source venv/bin/activate
python3 crontab_gen.py
```

When it asks for the report's `--out` path, point it at the repo root (`report.html`) — that's what `deploy/pi/docker-compose.yml` bind-mounts into the `caddy` container. Point ingestion (`runner.py`) at the `venv` on the SSD the same way it already runs on the dev machine.

7. Bring up API + Caddy

Built directly on the Pi — 64-bit Raspberry Pi OS is arm64, so this produces a native image with no cross-compilation needed:

```
cd /mnt/ebb-ssd/cb.ebb.claude.data/deploy/pi
docker compose build
docker compose up -d
```

If `docker/docker compose` aren't installed yet on the Pi, same install path as the dev machine: `sudo apt-get install -y docker.io docker-compose-v2`, then `sudo usermod -aG docker $USER` (needs a fresh login/`newgrp docker` to take effect — the plain `docker-compose v1` package has a known incompatibility with modern Docker Engine, hit this exact issue on the dev machine, use `docker-compose-v2` from the start here).

8. Firewall

Now genuinely internet-facing, not just LAN — basic defense in depth:

```
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
```

Verification

1. `docker compose ps` (from `deploy/pi/`) — both `api` and `caddy` show healthy.
2. `docker compose logs caddy` — confirms a cert was obtained for `ebb.hopto.org` (only works once port 443 actually reaches the Pi — see Prerequisites).
3. From outside the LAN (phone on cellular data, not Wi-Fi): `https://ebb.hopto.org/v1/meta/bounds` returns current archive stats.
4. `https://ebb.hopto.org/` loads the report.
5. Confirm a manual `python3 runner.py` run completes cleanly, then `python3 viewer_html.py --api http://127.0.0.1:8000 ...` regenerates `report.html` the same way it does on the dev machine.
6. Auth gate — same 4-case check used on the dev machine, against the Pi's own key from step 5:

```
curl -s -o /dev/null -w "%{http_code}\n" https://ebb.hopto.org/v1/meta/bounds #
curl -s -o /dev/null -w "%{http_code}\n" -X POST https://ebb.hopto.org/v1/messages \
  -H "Content-Type: application/json" -d '{"body":"test"}' #
curl -s -o /dev/null -w "%{http_code}\n" -X POST https://ebb.hopto.org/v1/messages \
  -H "Content-Type: application/json" -H "X-API-Key: $(grep EBB_API_KEY .env | cut -d= \
  -d '{"body":"test"}' #
```

Explicitly out of scope

- **Fixing the D-Link 443 issue itself** — hardware/router configuration, separate troubleshooting thread, tracked as the blocking prerequisite above.
- **Phase 5a optical archival export** (see `ROADMAP.md`) — tooling to bundle date ranges into ISO/BD-R images doesn't exist yet. The SSD here is just where a future export tool would eventually read *from* — not something this deployment builds.
- **DDNS updater** — `ebb.hopto.org` already resolves correctly (confirmed via `telnet` before writing this doc), assumed already maintained (eero or an existing No-IP client). If the Pi's public IP changes and nothing is updating `hopto.org`, that's a separate fix, not covered here.